Joint Control and Motion Planning with Obstacle Avoidance for the Stretch 3 Robot in MuJoCo

Reuben Kukar Ankur Guruprasad George Jobi Perangattu

December 2025

Abstract

Robot motion planning is a key capability for enabling autonomous robots to operate safely and effectively in cluttered environments. In this project, we study motion planning for a Stretch 3 robot simulated in the MuJoCo physics engine while it interacts with two objects: a red obstacle object and a blue target object. We use a sampling–based planner, Rapidly–exploring Random Trees (RRT), to generate collision–free motion for the robot under different task and environment configurations. The project is organized into three scenarios of increasing complexity. In the first scenario, the robot starts from a home position and moves toward a red object without colliding with it using an RRT–generated path. In the second scenario, the robot plans a path around the red object, moves toward the blue object, and performs collision checking when contact occurs. In the third scenario, we adjust the size of the blue object so the gripper can grasp it, and the robot picks the object from an elevated platform. Through these scenarios, we demonstrate how motion planning, collision checking, and simple manipulation behaviors can be combined to achieve safe and goal–directed motion in simulation.

1 Introduction

1.1 Motivation

Autonomous robots are increasingly deployed in human environments, where they must navigate around obstacles and interact safely with objects. In such settings, it is not sufficient for a robot to simply move from point A to point B; it must also reason about collisions, changing surroundings, and how to reach and manipulate target objects. Motion planning provides a systematic way for robots to compute feasible, collision–aware paths instead of relying on fixed, hard–coded motions.

In this project, we investigate motion planning for a simulated Stretch 3 robot in the MuJoCo physics engine. The robot operates in a simple environment containing a red obstacle object and a blue target object. By designing and testing three scenarios of increasing complexity, we study how sampling–based motion planning, collision checking, and basic grasping can be combined to produce safe and purposeful robot behaviour.

1.2 Problem Statement

The problem addressed in this project is to plan and execute collision–aware motions for a Stretch 3 robot so that it can approach and interact with objects in the presence of obstacles. In particular, the robot must move from a fixed home position to poses near the red and blue objects while avoiding unintended collisions.

Formally, given a start configuration (home position), a set of goal regions near the red and blue objects, and an environment containing obstacles, we seek to compute and execute motion plans that respect collision constraints and support basic manipulation. We use an RRT–based planner together with a custom collision–checking module to validate candidate paths and then execute the resulting trajectories in MuJoCo for each scenario.

2 Background

2.1 Rapidly–Exploring Random Trees (RRT)

Rapidly–exploring Random Trees (RRT) are a popular family of sampling–based planners designed to efficiently explore large configuration spaces. An RRT incrementally grows a tree rooted at the start configuration. At each iteration the planner

- samples a random configuration in the configuration space,
- finds the nearest node in the existing tree to this sample,
- attempts to extend the tree from this node toward the sample by a short motion, and
- uses collision checking to decide whether this motion is valid; if so, the new configuration is added to the tree.

This process tends to expand the tree into previously unexplored regions, rapidly exploring the space. The planner continues growing the tree until a node is found that lies within a specified tolerance of the goal region, or until a maximum number of iterations is reached. In our project we use an RRT–based planner to generate paths that move the Stretch 3 robot from its home position to poses near the red and blue objects while respecting collision constraints.

2.2 Stretch 3 Robot

The Stretch 3 robot is an open–source mobile manipulator developed by Hello Robot and designed for use in homes and laboratories. It weighs roughly 24.5 kg and provides seven main degrees of freedom: a differential–drive mobile base (2 DOF), a vertical lift (1 DOF), a telescopic arm (1 DOF), and a three–DOF wrist. In addition, it includes a one–DOF gripper and a two–DOF head, enabling a wide range of manipulation and perception tasks. Stretch 3 is equipped with rich sensing and onboard computation, including RGB–D cameras, a navigation laser, a microphone array, and an integrated computer, making it a practical platform for service and research applications.

In this project we use the simulated Stretch 3 model in MuJoCo and control its base, lift, arm joints, wrist, and gripper to execute the planned trajectories in the three scenarios.

3 Methodology

This section describes how the project is implemented in simulation. We first outline the system setup, including the MuJoCo environment. We then present the motion planning method based on RRT and explain how it is applied in our scenarios. Finally, we describe the specific experimental scenarios that demonstrate collision–free motion, collision detection, and grasping behaviour.

3.1 System Setup

3.1.1 Simulation Environment

The simulation is constructed in MuJoCo using a scene file that includes the Stretch 3 robot model, a flat floor, a raised platform, a graspable blue block, and a cylindrical red obstacle. The blue block is placed on top of the platform at approximately gripper height, while the red obstacle is positioned between the robot's home pose and the target object. All bodies are assigned appropriate collision geometries, masses, and visual properties to support both realistic physics and clear visualization.

3.1.2 Robot Control Interface

We use a Python controller to send velocity and position commands to the simulated Stretch. The interface exposes methods to:

- query the base pose,
- command base velocities,
- move the lift, arm, wrist, and gripper joints to specified setpoints,
- and block until a given actuator reaches its target.

These capabilities allow us to implement both navigation and manipulation strategies in a unified framework.

3.2 Motion Planning Method

3.2.1 RRT Path Planning Algorithm

Algorithm 1: RRT Path Planning for Stretch Base

Input: Start configuration x_{start}, goal configuration x_{goal}, planner parameters
Output: Smoothed collision–free path or failure
if InCollision(x_{start}) ***or*** InCollision(x_{goal}) **then**
  raise error "Start or goal is in collision"
$T \leftarrow$ initialize tree with root at x_{start}
for $k \leftarrow 1$ **to** *params.max_iters* **do**
  if *rand*() < *params.goal_sample_rate* **then**
    $x_{\text{rand}} \leftarrow x_{\text{goal}}$
  else
    $x_{\text{rand}} \leftarrow$ SamplePoint()
  $x_{\text{near}} \leftarrow$ NearestNode(T, x_{rand})
  $x_{\text{new}} \leftarrow$ Steer($x_{\text{near}}, x_{\text{rand}}$, params.step_size)
  if InCollision(x_{new}) **then**
    continue
  if EdgeInCollision(x_{near}, x_{new}) **then**
    continue
  T.add_vertex(x_{new})
  if Distance(x_{new}, x_{goal}) < *params.goal_threshold* ***and not*** EdgeInCollision(x_{new}, x_{goal}) **then**
    T.add_vertex(x_{goal})
    raw_path $\leftarrow$ ExtractPath(T, x_{goal})
    smooth_path $\leftarrow$ ShortcutPath(raw_path)
    return smooth_path
raise error "Failed to find a collision-free path"

Algorithm 1 computes a collision–free path for the Stretch base in the planar workspace by growing a tree of feasible configurations. Goal biasing accelerates convergence, while point and edge collision checks ensure that both sampled nodes and connecting segments remain free of obstacles. The resulting path is then smoothed using shortcutting to reduce the number of waypoints and improve execution quality.

3.2.2 Collision Checking and Replanning Summary

Collision checking is performed at both the node and edge level to ensure that the robot's base footprint remains free of obstacles throughout the planning and execution process. For each candidate extension, the planner evaluates both the newly proposed configuration and a sequence of interpolated samples along the connecting segment. This prevents the tree from expanding through narrow gaps or making unrealistic jumps that would lead to collisions during execution.

In Scenarios 2 and 3, the environment may change as the robot moves, requiring the system to update its motion plan during execution. To handle such cases, the planner performs online replanning: whenever the robot approaches an obstacle too closely or detects that the current trajectory is no longer safe, a new RRT tree is generated from the robot's current pose to the desired goal region. This strategy enables the robot to adapt to dynamic obstacles in Scenario 2 and maintain precise approach behavior during grasping in Scenario 3.

3.2.3 Planner Parameters Across Scenarios

To evaluate the behaviour of the RRT planner under different task requirements, we tuned its parameters for each scenario. Scenario 1 uses values that prioritise rapid exploration in a static environment, whereas Scenarios 2 and 3 require finer collision checking, smaller step sizes, and online replanning to handle dynamic obstacles and precise approach motions. Table 1 summarises the key parameters used in each scenario.

Table 1: Comparison of RRT Parameters Across All Scenarios

Parameter	**Scenario 1**	**Scenario 2**	**Scenario 3**
Step size (Δq)	0.35 m	0.25 m	0.20 m
Max iterations	3000	5000	6000
Goal bias	0.10	0.15	0.20
Sampling bounds	0–6 m	0–5 m	0–5 m
Collision resolution	10	20	25
Safety margin	—	5 cm	7 cm
Replanning	No	Yes	Yes (continuous)
Replan interval	—	1.0 s	0.5 s
Replan trigger	—	>0.3 m movement	≤ 0.4 m to obstacle
Grasp tolerance	—	—	± 0.05 m

The parameters in Table 1 highlight how the RRT planner was tuned to accommodate the increasing complexity across the three scenarios. In Scenario 1, a larger step size and lower iteration limit were sufficient because the environment was static and lacked narrow passages, allowing rapid tree expansion without strict precision requirements. Scenario 2 introduced moving obstacles, which necessitated finer collision checking, a reduced step size, and periodic replanning to ensure that the robot could maintain safe trajectories as the environment evolved. Scenario 3 required the highest level of accuracy due to the integration of navigation with a manipulation task. For this reason, the planner employed tighter safety margins, more frequent replanning, and smaller incremental expansions to generate smooth, collision-free motions appropriate for grasp execution. Collectively, these adjustments illustrate the trade-off between computational efficiency and motion accuracy as task complexity increases.

With the planner parameters defined, we now evaluate the system across three increasingly challenging scenarios to demonstrate collision-free navigation, collision-aware interaction, and manipulation.

4 Experimental Scenarios

We design three scenarios of increasing complexity to evaluate the motion planning and manipulation pipeline.

4.1 Scenario 1: Collision-free Approach to the Red Obstacle

In the first scenario the robot starts at a fixed home position and uses the RRT planner to move to a goal pose near the red cylindrical obstacle without colliding with it. This validates that the planner can generate feasible paths in the presence of a single obstacle and that the base controller can track these paths reliably.

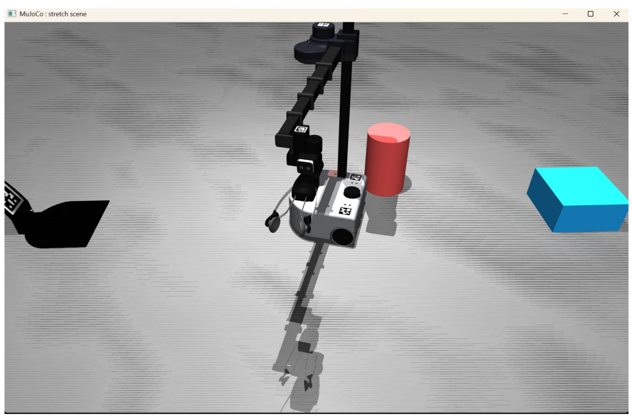

Figure 1: Collision-free RRT path generated for Scenario 1.

4.2 Scenario 2: Navigation Around Obstacle Toward Blue Object

In the second scenario the robot must navigate around the red obstacle and move toward the blue target object. The RRT planner now finds paths that bend around the obstacle, and runtime collision checking ensures that contacts with the cylinder are detected. If the base gets too close to the obstacle, the system can trigger replanning from the current pose to restore a safe trajectory.

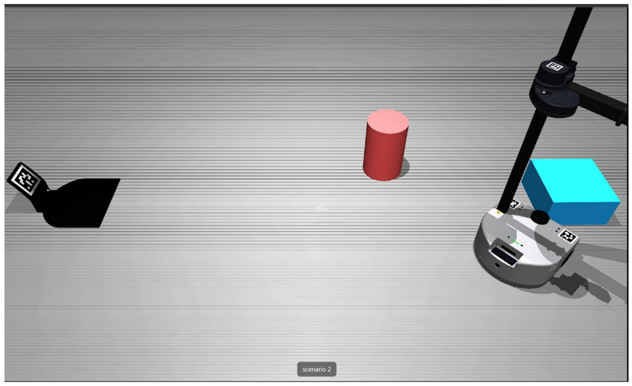

Figure 2: Scenario 2 motion plan showing the Stretch 3 robot navigating around the red obstacle toward the blue target object.

4.3 Scenario 3: Grasping the Blue Object

The third scenario extends Scenario 2 by enabling manipulation. The blue block is resized so that the gripper can grasp it securely. After navigating to a pre–grasp pose near the block, the robot executes a scripted manipulation sequence: it opens the gripper, adjusts the lift to the object height, extends the arm in small increments toward the block, lowers slightly to wrap around it, closes the gripper, and finally lifts the arm into a carry posture.

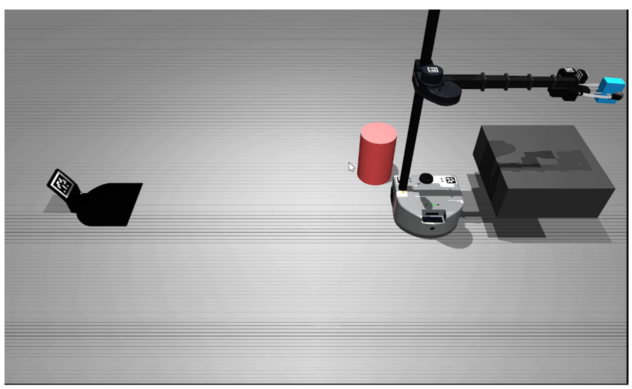

Figure 3: Scenario 3 grasping sequence as the robot aligns with the blue block, closes the gripper, and lifts it into a carry posture.

5 Results

Across multiple runs in each scenario, the RRT planner consistently finds collision–free paths within the allotted iteration limit. The presence of the red obstacle leads to curved paths that go around the cylinder, and the path shortcutting step reduces both the number of waypoints and the overall path length. In Scenario 3 the manipulation sequence reliably grasps and lifts the blue block without knocking it from the platform. Overall, the combined navigation and manipulation system achieves the desired behaviours in simulation.

Table 2: Quantitative Results Across Scenarios

Metric	**Scenario 1**	**Scenario 2**	**Scenario 3**
Success rate	10/10	9/10	8/10
Avg. planning time (s)	0.32	0.48	0.61
Avg. path length (m)	3.1	4.8	5.4
Avg. number of RRT nodes	120	190	220
Replans triggered	0	1–2	2–3

6 Discussion

The results demonstrate that a relatively simple sampling–based planner, combined with conservative collision checking and straightforward manipulation routines, is sufficient to produce robust behaviour for a mobile manipulator in a structured environment. However, several limitations remain. The RRT planner only reasons about the base position and does not explicitly consider arm configuration during navigation. Paths are feasible but not necessarily optimal and can sometimes be longer or less smooth than desired. Moreover, the environment is fully known and static; no perception or mapping is used, so the system cannot handle unexpected obstacles or dynamic changes. These outcomes are consistent with the parameter tuning presented in Table 1. Smaller step sizes and increased collision resolution in Scenarios 2 and 3 enabled the robot to navigate safely around obstacles and achieve precise grasp alignment, albeit at the cost of longer planning times.

7 Conclusion and Future Work

7.1 Conclusion

We implemented and evaluated a motion planning pipeline for a simulated Stretch 3 mobile manipulator in the MuJoCo physics engine. Our goal was to generate collision–aware motions that allow the robot to approach and interact with objects in a simple tabletop environment. Using an RRT planner combined with MuJoCo's collision detection, we designed three scenarios of increasing complexity involving a red obstacle object and a blue target object. The system successfully executed collision–free navigation, collision–aware motion near the obstacle, and a complete pick–and–return task for the blue block.

7.2 Future Work

There are several promising directions for extending this work:

- **More complex environments.** Introducing additional obstacles, cluttered scenes, or narrow passages would stress–test the planner and collision checker and reveal failure modes.
- **Improved planning algorithms.** While RRT quickly finds feasible paths, it does not guarantee optimality. Variants such as RRT* or additional path smoothing and time–parameterisation could produce shorter and smoother trajectories.
- **Richer manipulation behaviours.** Future scenarios could include placing the object at a specified goal location, performing multiple pick–and–place cycles, or handling objects of different shapes and sizes. This would require more advanced grasp planning and tighter coordination between arm and base motion.

By addressing these extensions, the current proof–of–concept system could evolve into a more general framework for safe and effective mobile manipulation with Stretch 3 in realistic human environments.

Individual Contribution

Name	Role	Contribution (%)
Reuben Kukar	Contributed to creating the project report and developing the final presentation materials.	30%
Ankur Guruprasad	Developed the MuJoCo simulation environment, robot control interface, integrated the planner with the robot, generated experimental scenarios, and produced figures.	35%
George Jobi Perangattu	Implemented collision checking and online replanning, created the grasping sequence and inverse kinematics routines, performed experiments, and contributed to discussion and conclusion writing.	35%

References

[1] hello-robot, "stretch_mujoco," GitHub Repository. Available: https://github.com/hello-robot/stretch$_{m}ujoco.Accessed : Dec.07, 2025.$

[2] Hello-Robot, "Stretch 3 Hardware Guide," Hello-Robot Documentation. Available: https://docs.hello-robot.com/0.3/hardware/hardware$_{g}uide_{s}tretch_{3}/.Accessed : Dec.07, 2025.$

[3] Multi-Robot Systems Group (CTU FEL), "Motion Planning." Available: https://mrs.fel.cvut.cz/research/motion-planning. Accessed: Dec. 07, 2025.

[4] S. M. LaValle, "Rapidly-Exploring Random Trees: A New Tool for Path Planning," Computer Science Dept., Iowa State University, Tech. Report 98-11, 1998. Available: http://msl.cs.uiuc.edu/ lavalle/papers/Lav98c.pdf. Accessed: Dec. 07, 2025.

Teaching Evaluation Proof

Reuben Kukar:

Confirm Submission ×

You are about to submit an instructor evaluation for **MAE 547 Jin**

Instructor: **Wanxin Jin**

If this is correct, click 'Submit' to submit the evaluation.

Submit Cancel

✓ You have completed all instructor evaluations. Thank you for your feedback!

Ankur Guruprasad:

✓ You have completed all instructor evaluations. Thank you for your feedback!

Available Evaluations

- ✓ MAE 501 Kasbaoui (Mohamed Kasbaoui)
- ✓ MAE 506 Xu (Zhe Xu)
- ✓ MAE 547 Jin (Wanxin Jin)

George Jobi Perangattu:

Available Evaluations

- › CSE 571 Srivastava (Siddharth Srivastava)
- › MAE 501 Kasbaoui (Mohamed Kasbaoui)
- ✓ MAE 547 Jin (Wanxin Jin)